X

(https://swayam.gov.in) (https://swayam.gov.in/nc_details/NPTEL)

pradipmaity9907@gmail.com

NPTEL (https://swayam.gov.in/explorer?ncCode=NPTEL) » The Joy of Computing using Python (course)

Course outline

About NPTEL ()

How does an NPTEL online course work? ()

Week 1 ()

Week 2 ()

Week 3 ()

week 4 ()

Week 5 ()

Week 6 ()

Week 7 ()

Week 8 ()

Week 9 ()

Natural Language Processing - Author Stylometry (unit?unit=188&lesson=189)

Natural Language Processing - Author Stylometry - Part 01 (unit?unit=188&lesson=190)

Natural Language Processing - Author Stylometry

Week 9 : Assignment 9

The due date for submitting this assignment has passed.

Due on 2026-03-25, 23:59 IST.

Assignment submitted on 2026-03-23, 11:26 IST

Stylometric and Sentiment Analysis of the Federalist Papers

Background

A political science research team is examining the **Federalist Papers**, a historically significant collection of essays written to support the ratification of the U.S. Constitution. While many essays are clearly attributed to **Hamilton, Madison,** or **Jay,** a subset of essays has **disputed or shared authorship**.

Instead of relying on historical arguments alone, the team decides to use **computational text analysis** to study writing patterns across different author groups.

Problem Statement

The team wants to understand whether **quantitative textual characteristics** , such as word usage patterns and sentiment , can help differentiate between authors and provide insights into disputed essays.

The task is **not to assign authorship directly**, but to compare:

- Writing style indicators
- Distributional properties of words
- Emotional tone across author groups

Dataset Description

The dataset is organized by author category:

- **Hamilton**
- **Madison**
- **Jay**
- **Disputed**
- **Shared**

Each category contains a block of text representing essays attributed to that group. These texts are treated as independent corpora.

- Part 02 (unit?unit=188&lesson=191)
Natural Language Processing - Author Stylometry - Part 03 (unit?unit=188&lesson=192)
Natural Language Processing - Author Stylometry - Part 04 (unit?unit=188&lesson=193)
Natural Language Processing - Author Stylometry - Part 05 (unit?unit=188&lesson=194)
Natural Language Processing - Author Stylometry - Part 06 (unit?unit=188&lesson=195)
Natural Language Processing - Author Stylometry - Part 07 (unit?unit=188&lesson=196)
Natural Language Processing - Author Stylometry - Part 08 (unit?unit=188&lesson=197)
Natural Language Processing - Author Stylometry - Part 09 (unit?unit=188&lesson=198)
Natural Language Processing - Author Stylometry - Part 10 (unit?unit=188&lesson=199)
Introduction to Networkx - Part 01 (unit?unit=188&lesson=200)
Introduction to Networkx - Part 02 (unit?

Analytical Approach

The analysis follows four major stages:

1. Text Preprocessing

Each author's text is processed to extract meaningful words by:

- Tokenizing text into individual words
- Removing punctuation and non-alphabetic tokens
- Eliminating common English stopwords

This ensures that comparisons focus on **content-bearing words** rather than grammatical fillers.

2. Stylometric Analysis Using Word Length

For every author group:

- The length of each word is computed
- A frequency distribution of word lengths is constructed

Word length is treated as a **stylistic signature**, allowing comparison of writing habits without relying on vocabulary meaning.

3. Sentiment Analysis

Each author's entire text is analyzed using a sentiment model to compute:

- A **polarity score** ranging from negative to positive

This helps identify whether different authors (or disputed texts) consistently express different emotional tones, even within formal political writing.

4. Visual Comparison

Word-length frequency distributions are plotted for all authors on the same figure.
This visual representation allows analysts to:

- Observe overlaps between authors
- Identify stylistic similarities
- Spot anomalies in disputed or shared texts

unit=188&lesson=201)
- Six Degrees of Separation : Meet your favourites (unit?unit=188&lesson=202)
- Six Degrees of Separation : Meet your favourites - Part 01 (unit?unit=188&lesson=203)
- Six Degrees of Separation : Meet your favourites - Part 02 (unit?unit=188&lesson=204)
- Six Degrees of Separation : Meet your favourites - Part 03 (unit?unit=188&lesson=205)
- Area Calculation - Don't Measure (unit?unit=188&lesson=206)
- Area Calculation - Don't Measure - Part 01 (unit?unit=188&lesson=207)
- Area Calculation - Don't Measure - Part 02 (unit?unit=188&lesson=208)
- Area Calculation - Don't Measure - Part 03 (unit?unit=188&lesson=209)
- Area Calculation - Don't Measure - Part 04 (unit?unit=188&lesson=210)
- Area Calculation - Don't Measure - Part 05 (unit?unit=188&lesson=211)
- Area Calculation - Don't Measure -

```python
import nltk
from nltk.tokenize import word_tokenize
from nltk.corpus import stopwords
from nltk.probability import FreqDist
import matplotlib.pyplot as plt
from textblob import TextBlob


authors = ["Hamilton", "Madison", "Disputed", "Jay", "Shared"]

federalist_by_author = {
    "Hamilton": "Text data for Hamilton...",
    "Madison": "Text data for Madison...",
    "Disputed": "Text data for Disputed...",
    "Jay": "Text data for Jay...",
    "Shared": "Text data for Shared..."
}


author_tokens = {}
for author, text_data in federalist_by_author.items():
    tokens = [
        word for word in word_tokenize(text_data)
        if word.isalpha() and word.lower() not in stopwords.words("english")
    ]
    author_tokens[author] = tokens
```

```python
author_word_lengths = {}
for author, tokens in author_tokens.items():
    word_lengths = [len(word) for word in tokens]
    author_word_lengths[author] = word_lengths


sentiment_scores = {}
for author, text_data in federalist_by_author.items():
    blob = TextBlob(text_data)
    sentiment_scores[author] = blob.sentiment.polarity


plt.figure(figsize=(12, 6))
for author in authors:
    fdist = FreqDist(author_word_lengths[author])
    fdist.plot(15, cumulative=False)
plt.legend(authors)
plt.show()

for author, sentiment in sentiment_scores.items():
    print(f"Sentiment analysis for {author}: Sentiment Score = {sentiment}")
```

1) Assume the text for *Hamilton* contains only the words: "Liberty liberty the Constitution" ***1 point***

After preprocessing, what will be stored in author_tokens["Hamilton"]?

- ["Liberty", "liberty", "Constitution"]
- ["Liberty", "liberty", "the", "Constitution"]

Part 06 (unit?unit=188&lesson=212)
Quiz: Week 9 : Assignment 9 (assessment?name=659)
Week 9 : Programming Assignment 1 (/noc26_cs84/progassignment?name=640)
Week 9 : Programming Assignment 2 (/noc26_cs84/progassignment?name=641)
Week 9 : Programming Assignment 3 (/noc26_cs84/progassignment?name=642)
Week 9 Feedback Form: The Joy of Computing using Python (unit?unit=188&lesson=213)
Week 10 ()
Week 11 ()
Week 12 ()
Text Transcripts ()
Download Videos ()

["liberty", "liberty", "constitution"]
["Liberty", "Constitution"]

Yes, the answer is correct.
Score: 1
Accepted Answers:
["Liberty", "liberty", "Constitution"]

2) If an author's text contains **only stopwords**, what happens when plotting their word-length distribution? ***1 point***

- The program raises a runtime error
- An empty plot is generated without bars
- All word lengths are plotted as zero
- The author is skipped automatically

Yes, the answer is correct.
Score: 1
Accepted Answers:
An empty plot is generated without bars

3) Which author property is **directly visualized** in the plotted graphs? ***1 point***

- Sentiment polarity
- Token frequency
- Word-length frequency
- Vocabulary richness

Yes, the answer is correct.
Score: 1
Accepted Answers:
Word-length frequency

4) If the *Disputed* corpus contains significantly longer average words than others, how would this most likely appear in the plot? ***1 point***

- Higher bars at smaller word lengths
- A wider spread with peaks at larger word lengths
- A flat distribution across all lengths
- No visible difference from other authors

Yes, the answer is correct.
Score: 1
Accepted Answers:
A wider spread with peaks at larger word lengths

5) Suppose TextBlob returns a sentiment polarity of 0.0 for *Jay.* What does this imply? ***1 point***

- Jay's text is emotionally neutral overall
- Jay uses no adjectives
- Jay's text contains no positive words
- Jay's text failed sentiment analysis

Yes, the answer is correct.
Score: 1
Accepted Answers:
Jay's text is emotionally neutral overall

6) If the stopword filter were removed from preprocessing, which change would MOST likely occur? ***1 point***

- Sentiment scores become negative
- Word-length distributions shift toward shorter lengths

- Vocabulary size decreases
- Alphabetic filtering stops working

Yes, the answer is correct.
Score: 1
Accepted Answers:
Word-length distributions shift toward shorter lengths

7) Why is sentiment analysis performed on the **entire text** rather than token-level sentiment? ***1 point***

- TextBlob does not support token sentiment
- Corpus-level sentiment smooths local noise
- Token sentiment would always be zero
- Tokens are removed before sentiment analysis

Yes, the answer is correct.
Score: 1
Accepted Answers:
Corpus-level sentiment smooths local noise

8) If the *Shared* corpus was written by multiple authors, what pattern would MOST likely appear? ***1 point***

- A sharply peaked word-length distribution
- Identical sentiment to all authors
- A broader, less consistent distribution
- No visible plot

Yes, the answer is correct.
Score: 1
Accepted Answers:
A broader, less consistent distribution

Exploring Social Networks Using Shortest Paths

Background

A data analyst is interested in understanding how people are connected within a large online social network. To do this, the analyst works with a real-world dataset representing friendships between users on a social media platform. Each user is modeled as a **node**, and each friendship is modeled as an **edge** in a graph.

The analyst chooses **NetworkX**, a Python library designed for creating, analyzing, and studying complex networks, to perform this analysis.

Problem Context

One of the most well-known ideas in social network analysis is the **Six Degrees of Separation** theory, which suggests that any two people in the world are connected by a surprisingly small number of intermediate connections.

The analyst wants to verify whether this idea holds for the given social network dataset by answering the following questions:

- How closely connected are users on average?
- What is the shortest possible connection between any two users?
- What is the longest shortest path observed in the network?
- Does the network show properties that support the idea of six degrees of separation?

Data Description

- The dataset is stored in a text file (facebook_combined.txt).
- Each line represents a friendship between two users.
- The graph is **undirected**, meaning friendships work both ways.

Analytical Approach

Using the provided code, the analyst performs the following steps:

1. Network Construction

o The dataset is read and converted into a graph structure.
o All users (nodes) are extracted for analysis.

2. Shortest Path Computation

o For every pair of distinct users, the shortest path length is calculated.
o These values represent the minimum number of connections required to move from one user to another.

3. Statistical Analysis of Paths

o All shortest path lengths are stored.
o The minimum, maximum, and average shortest path lengths are computed.

4. Interpretation of Results

o The **minimum shortest path** shows the closest possible connection between any two users.
o The **maximum shortest path** indicates the farthest distance between two users while still remaining connected.
o The **average shortest path length** serves as the most important metric for validating the Six Degrees of Separation theory.

Key Observations

- If the **average shortest path length** is close to 6 (or lower), it supports the idea that people in the network are closely connected.
- A relatively small **maximum shortest path length** suggests that even the most distant users are not too far apart.
- These findings highlight how social networks enable rapid connectivity through indirect relationships.

Conceptual Insights

- **Six Degrees of Separation:** The experiment demonstrates how short paths emerge naturally in large social networks.
- **Exponential Growth of Connections:** As each user has multiple friends, and each friend has their own friends, the number of reachable users grows exponentially with each step.
- **Graph Theory in Practice:** Abstract concepts like nodes, edges, and shortest paths directly translate into real-world social phenomena.

```python
import networkx as nx

G = nx.read_edgelist("facebook_combined.txt")
N = list(G.nodes)

shortest_path_length_list = []
for u in N:
    for v in N:
        if u != v:
            l = nx.shortest_path_length(G, source=u, target=v)
            print(f"Shortest path between {u} and {v} is of length {l}")
            shortest_path_length_list.append(l)

minimum_shortest_path_length = min(shortest_path_length_list)
maximum_shortest_path_length = max(shortest_path_length_list)
average_shortest_path_length = sum(shortest_path_length_list) / len(shortest_path_length_list)

print("Minimum Shortest Path Length:", minimum_shortest_path_length)
print("Maximum Shortest Path Length:", maximum_shortest_path_length)
print("Average Shortest Path Length:", average_shortest_path_length)
print("Six Degrees of Separation: The idea that you can connect with anyone in the world through a network of acquaintances within an average of six connectior
print("Exponential Growth in Connections: This concept works because of the exponential growth in the number of friends' friends within a social network.")
print("Encouragement to Explore NetworkX: You can explore the NetworkX library further and make use of its various functionalities for graph analysis.")
```

Code for Reference

9) What is the **primary purpose** of using nx.read_edgelist("facebook_combined.txt") in the case study? ***1 point***

- (●) To create a graph where users are nodes and friendships are edges
- ( ) To calculate shortest paths directly from the file
- ( ) To store shortest path lengths in a list
- ( ) To visualize the social network

Yes, the answer is correct.
Score: 1

Accepted Answers:
To create a graph where users are nodes and friendships are edges

10) Why does the code check if u != v before computing the shortest path? ***1 point***

- ( ) To prevent duplicate edges in the graph
- (●) To avoid self-loops that always have zero path length
- ( ) To reduce memory usage
- ( ) To ensure the graph remains undirected

Yes, the answer is correct.
Score: 1

Accepted Answers:
To avoid self-loops that always have zero path length

11) What does the list shortest_path_length_list primarily represent? ***1 point***

- ( ) The degree of each node
- ( ) The number of friends per user
- (●) All shortest path lengths between distinct pairs of nodes
- ( ) Only the longest path in the network

Yes, the answer is correct.
Score: 1

Accepted Answers:
All shortest path lengths between distinct pairs of nodes

12) Which metric from the code is **most relevant** for validating the Six Degrees of Separation concept? ***1 point***

- Minimum shortest path length
- Maximum shortest path length
- Number of nodes in the graph
- Average shortest path length

Yes, the answer is correct.
Score: 1

Accepted Answers:
Average shortest path length

13) If the network is **fully connected**, what would most likely happen to the maximum shortest path length? ***1 point***

- It would increase significantly
- It would become zero
- It would remain relatively small
- It would become equal to the number of nodes

Yes, the answer is correct.
Score: 1

Accepted Answers:
It would remain relatively small

14) Why does the case study mention **exponential growth in connections**? ***1 point***

- Because shortest path lengths grow exponentially
- Because the number of nodes doubles each iteration
- Because friend-of-friend connections expand rapidly with each step
- Because NetworkX uses exponential algorithms internally

Yes, the answer is correct.
Score: 1

Accepted Answers:
Because friend-of-friend connections expand rapidly with each step

15) What is a **computational limitation** of the approach used in the code? ***1 point***

- It cannot handle undirected graphs
- It only works for weighted graphs
- It ignores isolated nodes
- It computes shortest paths for every node pair, which is expensive

Yes, the answer is correct.
Score: 1

Accepted Answers:
It computes shortest paths for every node pair, which is expensive